Корреляция и линейная регрессия

Конспект по темам 4–5. Всё, что нужно для решения практики: от смысла коэффициента корреляции до интерпретации параметров модели в четырёх формах с логарифмами.

1. Корреляция: что это

Корреляция — мера статистической связи между двумя признаками. Принимает значения от −1 до 1.

```
−1 ──────────── −0.1 ── 0 ── 0.1 ──────────── 1
СИЛЬНАЯ              ОТСУТСТВИЕ СВЯЗИ          СИЛЬНАЯ
```

- **Отрицательная (обратная):** если A растёт, B убывает; если A убывает, B растёт.
- **Положительная (прямая):** если A растёт, B тоже растёт.
- **От −0.1 до 0.1:** связь практически отсутствует.

> **Главное правило интерпретации.** Корреляция не говорит о причинно-следственной связи (correlation is not causation). Слова **«влияет», «зависит», «причина», «приводит к»** в выводе НЕКОРРЕКТНЫ. Корректно: «существует связь», «связаны», «чем больше A, тем больше B», «это может повысить / понизить».

Два коэффициента

	Пирсон	Спирман
Что измеряет	линейную связь	монотонную связь
Между какими признаками интерпретируется	только между количественными	между количественными и категориальными
Выбросы	чувствителен	устойчив
Код	`df['A'].corr(df['B'])`	`df['A'].corr(df['B'], method='spearman')`

Монотонность — ключ к различию

Монотонная зависимость — та, где признак всё время растёт либо всё время убывает, но не обязательно по прямой: линейная, степенная, квадратный корень, арксинус. **Немонотонная** — где направление меняется: квадратичная (парабола), циклическая (синусоида).

- Связь линейная → Пирсон и Спирман близки.
- Связь монотонная, но изогнутая → **Спирман заметно больше Пирсона по модулю**. Вывод: связь скорее нелинейная.
- Связь немонотонная → оба коэффициента около нуля, хотя связь в данных есть.

Корреляционная матрица

```
df.corr(numeric_only=True)                    # Пирсон
df.corr(numeric_only=True, method='spearman') # Спирман
```

По центральной диагонали стоят единицы — это **автокорреляция**, корреляция признака с самим собой. Матрица симметрична относительно диагонали.

Чтобы найти признак с самой сильной связью с целевой — извлечь столбец и отсортировать **без учёта знака**:

```
cm = df.corr(method='spearman')
cm['selling_price'].sort_values(key=abs)
```

`key=abs` сортирует по модулю: сила связи не зависит от направления.

2. Кодирование категорий перед корреляцией

Корреляцию нельзя посчитать с текстовым признаком, поэтому категории переводят в числа.

Способ	Код	Что получается
One-Hot	`pd.get_dummies(df, columns=['fuel'], dtype='int')`	отдельный столбец 0/1 на каждую категорию: `fuel_Petrol`, `fuel_Diesel` …
LabelEncoder	`LabelEncoder().fit_transform(df['transmission'])`	один столбец с целыми числами по алфавиту: Automatic = 0, Manual = 1

`dtype='int'` нужен, чтобы получить 1/0, а не True/False.

После One-Hot каждый новый столбец бинарный, и его корреляция с целевой читается так: *если автомобиль на бензине (1), это может понизить его стоимость*. Если коэффициент около нуля (например 0.09) — **монотонная связь отсутствует**, интерпретировать нечего.

3. Как формулировать вывод по корреляции

Шаблон из трёх элементов: **сила + направление + тип связи**.

Значение	Сила
0.0 – 0.1	связь практически отсутствует
0.1 – 0.3	слабая
0.3 – 0.5	умеренная
0.5 – 0.7	умеренная, близкая к сильной
0.7 – 1.0	сильная

Примеры правильных формулировок:

- Пирсон 0.42 → «умеренная прямая линейная взаимосвязь».
- Спирман −0.75 → «сильная обратная монотонная взаимосвязь».
- Пирсон 0.002 → «линейная связь практически отсутствует» (но не «связи нет вообще»: нелинейная может быть).
- Пирсон −0.19, Спирман −0.31 → «слабая обратная линейная связь, при этом возможна нелинейная взаимосвязь» — потому что Спирман по модулю заметно больше.

Ловушки в тестах. Неверны формулировки вида «Пирсон = 0.42, значит умеренная прямая *нелинейная* связь» (Пирсон измеряет линейную) и «причиной низкой цены является год выпуска» (причинность). Верна всегда фраза «мы не можем утверждать, что причиной X является Y».

4. Линейная регрессия: устройство

Линейная регрессия — метод моделирования зависимости между одним или несколькими **предикторами** (независимыми переменными) и количественной **целевой переменной**.

$$Y = w_0 + w_1 \cdot X_1 + \ldots + w_n \cdot X_n$$

- w_0 — свободный коэффициент;
- $w_1 \ldots w_n$ — коэффициенты независимых переменных;
- одна X → однофакторная регрессия, несколько → многофакторная.

Подбор предикторов

- Берём X с самой сильной связью с Y — смотрим корреляционную матрицу.
- Между самими предикторами сильной корреляции избегаем: это **мультиколлинеарность**, предикторы дублируют друг друга.
- Минимум 100 наблюдений на каждый предиктор.
- Предпочтительны количественные признаки с симметричным куполообразным распределением. Несимметричные по возможности логарифмируем.

Деление выборки и обучение

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X = df[['предиктор 1', 'предиктор 2', ...]]
Y = df['целевая']

X_train, X_test, y_train, y_test = train_test_split(X, Y,
                                                    test_size=0.2,
                                                    random_state=42)

model = LinearRegression().fit(X_train, y_train)
model.intercept_   # w0
model.coef_        # массив [w1, w2, ...] в порядке столбцов X
```

Тренировочная выборка (80%) нужна для обучения — поиска коэффициентов методом наименьших квадратов. Тестовая (20%) — для честной оценки качества. `random_state` фиксирует случайное разбиение, чтобы результат воспроизводился.

5. Интерпретация параметров — главная тема

Как читается коэффициент, зависит от того, что логарифмировано. Четыре формы:

Форма модели	X количественный	X категориальный / бинарный
$Y = w_0 + w \cdot X$	X вырос на 1 → Y меняется на **w** единиц	X равен 1 → Y меняется на **w** единиц
$Y = w_0 + w \cdot \ln(X)$	X вырос на 1% → Y меняется на **w / 100** единиц	—
$\ln(Y) = w_0 + w \cdot X$	X вырос на 1 → Y меняется на **100·w %**	X равен 1 → Y меняется на **100·w %**
$\ln(Y) = w_0 + w \cdot \ln(X)$	X вырос на 1% → Y меняется на **w %**	—

Знак читается прямо: положительный коэффициент — рост, отрицательный — снижение. Направление «снизится на 1» даёт зеркальный вывод: если w = −0.7 и X снизится на 1, Y увеличится на 0.7.

Свободный коэффициент

- В обычной модели: если все предикторы равны 0, то $Y = w_0$.
- В модели с ln(Y): если все предикторы равны 0, то $Y = e^{w_0}$.
- Интерпретировать его можно не всегда — часто сочетание «все предикторы равны нулю» в данных не встречается.

Разбор на числах

Модель	Коэффициент	Как читается
Y = … + w·X, X количественный	w = −0.7	X вырос на 1 → цена ниже на 0.7 доллара. X снизился на 1 → цена выше на 0.7
Y = … + w·X, X бинарный	w = 51 773	X равен 1 → цена выше на 51.77 тыс. долларов
Y = … + w·ln(X)	w = −55 714	X вырос на 1% → цена ниже на 557.14 доллара
ln(Y) = … + w·X, X бинарный	w = −0.4941	X равен 1 → это снижает цену на 49.41%
ln(Y) = … + w·X, X количественный	w = 0.1144	X вырос на 1 → цена выше на 11.44%
ln(Y) = … + w·ln(X)	w = −0.0315	X вырос на 1% → цена ниже на 0.03%

Что отличает верный ответ от неверного в тестах. Путаница «на 1» против «на 1%» и «единиц» против «процентов» — именно на этом строятся варианты ответов. Сначала определите форму модели (что под логарифмом), потом тип X, и только потом смотрите на число.

6. Предсказание

```
model.predict([[значение X1, значение X2, ...]])
```

Значения подставляются в том же порядке, в каком столбцы шли в X. Часто в задании просят подставить не конкретные числа, а статистики:

- «самая частая категория» → `df['X'].mode()[0]`;
- «самая редкая категория» → `df['X'].value_counts().idxmin()`;
- «первый квартиль» → `df['X'].quantile(0.25)`;
- «медиана» → `df['X'].median()`;
- «максимум» / «минимум» → `.max()` / `.min()`.

Если целевая логарифмирована, `predict` вернёт ln(Y), и результат нужно вернуть обратно экспонентой:

```
import numpy as np
np.exp(model.predict([[...]])[0])
```

7. Метрики качества

Считаются **на тестовой выборке**.

```
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

y_pred = model.predict(X_test)
r2_score(y_test, y_pred)
mean_squared_error(y_test, y_pred)
mean_absolute_error(y_test, y_pred)
```

Метрика	Что описывает	Значения
R^2 (коэффициент детерминации)	долю разброса целевой переменной, объяснённую моделью	от 0 до 1 (редко отрицательный); чем ближе к 1, тем лучше
MSE (среднеквадратичная ошибка)	усреднённый квадрат ошибки	от 0 до ∞; меньше — лучше; для сравнения моделей
MAE (средняя абсолютная ошибка)	усреднённую ошибку в единицах Y	от 0 до ∞; меньше — лучше

Как читать:

- $R^2 = 0.38$ → модель объясняет 38% вариации, **62% остались не описаны моделью**.
- MAE = 215 914 → в среднем модель ошибается более чем на 215 тыс. долларов. Именно MAE читается содержательно.
- MSE = 198 734 432 876 → это доллары в квадрате, абсолютное значение не интерпретируется. Используется только для сравнения.
- Разница MAE двух моделей = насколько одна точнее другой: 215 914.58 − 214 619.12 = **1 295.46 доллара**.

Три ловушки. (1) «MSE больше MAE, значит модель хорошая» — бессмыслица, это разные метрики в разных единицах, и MSE почти всегда больше арифметически. (2) «В среднем модель ошибается на MSE» — нет, на MAE. (3) MSE и MAE можно сравнивать **только между моделями с одинаковой целевой переменной**: если у одной Y в долларах, а у другой ln(Y), их ошибки несопоставимы. R^2 сравнивать можно всегда.

Ещё одно полезное утверждение: если данные не очищены от выбросов, **MAE более подходящая метрика, чем MSE** — квадрат сильнее штрафует крупные промахи, а они как раз создаются выбросами.

8. Логарифмирование

Деньги и пробеги почти всегда скошены вправо: много мелких значений и длинный хвост крупных. Логарифм сжимает большие значения сильнее маленьких, и распределение становится похожим на колокол.

```
import numpy as np
df['ln_price'] = np.log(df['selling_price'])
np.exp(значение)   # обратно
```

Как распознать на гистограммах. ДО логарифмирования — один высокий столбик у левого края и почти пустая полоса вправо. ПОСЛЕ — либо симметричный колокол, либо распределение со скосом влево (если в данных есть экстремально малые значения, например пробег 1 км: после логарифма они уходят далеко влево).

Практический эффект: в примере с автомобилями замена пробега на ln(пробег) подняла R^2 с 0.38 до 0.39, а логарифмирование ещё и целевой переменной — до 0.67.

9. Шпаргалка формулировок

В задании спрашивают	Что делать
«степень линейной взаимосвязи»	корреляция Пирсона, `.corr()`
«самая сильная монотонная связь»	Спирман, сортировка по модулю
«самая сильная обратная связь»	самое отрицательное значение
«самая сильная прямая связь»	самое положительное значение
«отсутствует монотонная связь»	коэффициент в пределах ±0.1
«среднее целевой по тестовой выборке»	`y_test.mean()`
«какая будет стоимость, если…»	`model.predict([[...]])`, при ln(Y) — ещё `np.exp`
«сколько % вариации не описано»	$(1 - R^2) \cdot 100$
«на сколько в среднем ошибается»	MAE